

Experiment 1: Development of a Chat Application Based on Socket Programming

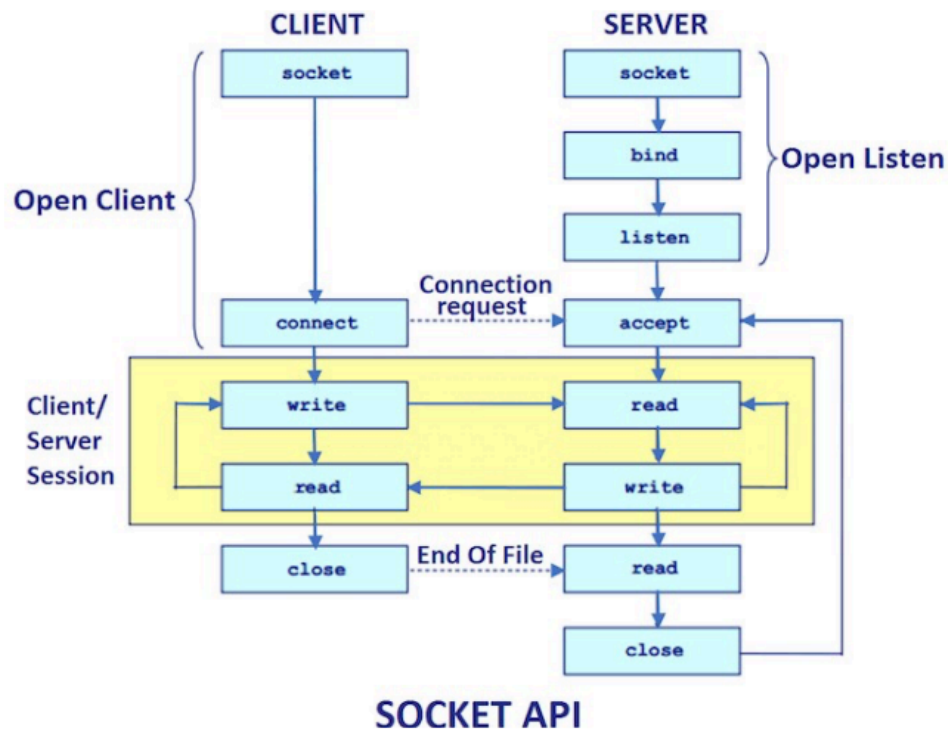
AIM:

To Develop a Chat Application Based on Socket Programming.

PROCEDURE:

1. Create separate Java GUI applications for the **Server** and **Client** using Java Swing components such as JFrame, JTextArea, JTextField, and JButton.
2. Implement socket programming using ServerSocket for the server and Socket for the client to establish communication.
3. Compile both the server and client programs using the Java compiler.
4. Execute the server application first and then run the client application to establish a connection.
5. Type messages in the text field and send them using the **Send** button. The messages are displayed in the chat area of both applications.
6. Continue exchanging messages between the server and client until the conversation is completed or terminated.
7. Close both applications after successful communication and verify that the messages are exchanged correctly through the GUI.

FLOW CHART:



CODE:

ServerGUI.java

```
import javax.swing.*;
import java.awt.*;
import java.net.*;
import java.io.*;

public class ServerGUI {
    public static void main(String[] args) {
        try {
            // ----- Socket -----
            ServerSocket server = new ServerSocket(5000);
            System.out.println("Server Started...");
            Socket socket = server.accept();
            System.out.println("Client Connected!");
            BufferedReader in =
                new BufferedReader(
                    new InputStreamReader(
                        socket.getInputStream()));
            PrintWriter out =
                new PrintWriter(
                    socket.getOutputStream(), true);

            // ----- GUI -----
            JFrame frame = new JFrame("Server Chat");
            frame.setSize(500, 500);
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setLocationRelativeTo(null);
            JTextArea chatArea = new JTextArea();
            chatArea.setEditable(false);
            JScrollPane scroll = new JScrollPane(chatArea);
            JTextField input = new JTextField(25);
            JButton send = new JButton("Send");

            JPanel panel = new JPanel();
            panel.add(input);
            panel.add(send);
            frame.add(scroll, BorderLayout.CENTER);
```

```

frame.add(panel, BorderLayout.SOUTH); //bottom panel
frame.setVisible(true);

// ----- Send Button -----
send.addActionListener(e -> {
    String msg = input.getText();
    out.println(msg); //sends msges to the client
    chatArea.append("Server : " + msg + "\n"); //displays the msg sent
    input.setText("");
});
// ----- Receive -----
while (true) {
    String msg = in.readLine(); //reads msges from client
    if (msg == null)
        break;
    chatArea.append("Client : " + msg + "\n"); //displays the received msges
    if (msg.equalsIgnoreCase("bye"))
        break;
}
socket.close();
server.close();
} catch (Exception e) {
    System.out.println(e);
}
}
}

```

ClientGUI.java

```

import javax.swing.*.*;
import java.awt.*.*;
import java.net.*;
import java.io.*;

public class ClientGUI {

    public static void main(String[] args) {

        try {

            Socket socket = new Socket("localhost", 5000);

```

```

BufferedReader in =
    new BufferedReader(
        new InputStreamReader(
            socket.getInputStream()));
PrintWriter out =
    new PrintWriter(
        socket.getOutputStream(), true);

JFrame frame = new JFrame("Client Chat");
frame.setSize(500, 500);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setLocationRelativeTo(null);

JTextArea chatArea = new JTextArea();
chatArea.setEditable(false);

JScrollPane scroll = new JScrollPane(chatArea);

JTextField input = new JTextField(25);
JButton send = new JButton("Send");

JPanel panel = new JPanel();
panel.add(input);
panel.add(send);

frame.add(scroll, BorderLayout.CENTER);
frame.add(panel, BorderLayout.SOUTH);

frame.setVisible(true);

send.addActionListener(e -> {
    String msg = input.getText(); //sends msgs to server
    out.println(msg);
    chatArea.append("Client : " + msg + "\n");//displays the msg sent
    input.setText("");
});

while (true) {
    String msg = in.readLine(); //reads msges from server
    if (msg == null)
        break;
    chatArea.append("Server : " + msg + "\n"); //displays the received msges
    if (msg.equalsIgnoreCase("bye"))
        break;
}

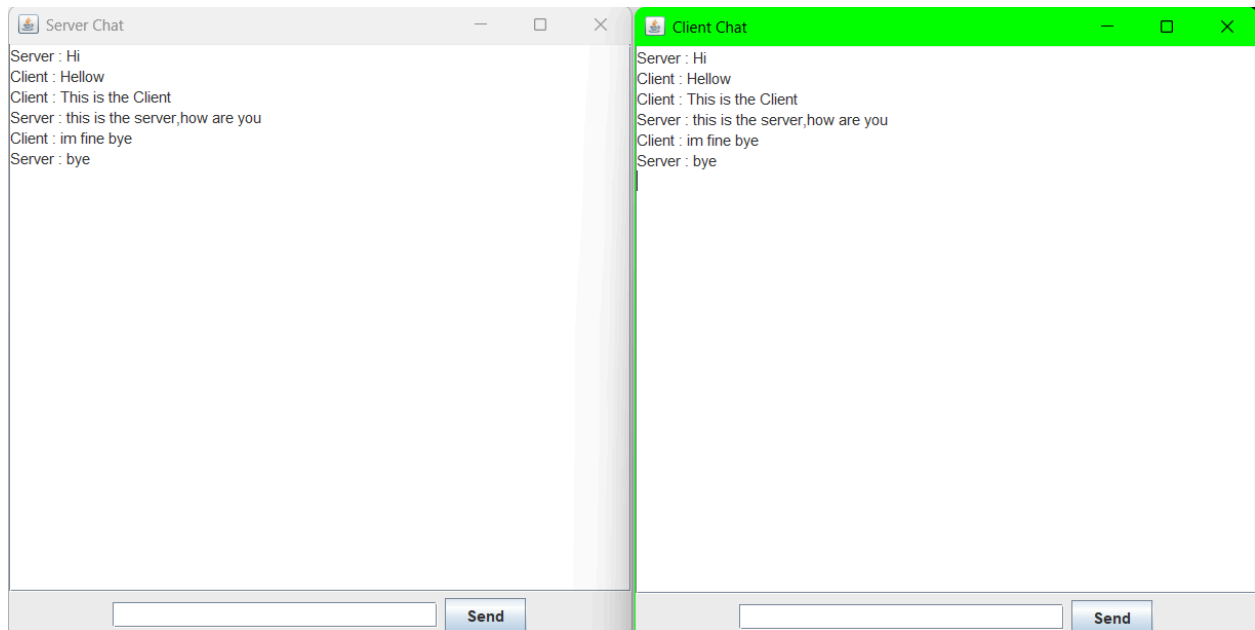
```

```

    }
    socket.close();
} catch (Exception e) {
    System.out.println(e);
}
}
}

```

OUTPUT:



RESULTS:

A GUI-based chat application was successfully developed using Java Swing and Socket Programming. The server and client established a connection successfully, and messages were exchanged through the graphical interface as expected.

INFERENCE:

This experiment demonstrated the implementation of a GUI-based client-server chat application using Java Socket Programming and Swing. It provided an understanding of establishing TCP connections, exchanging messages in real time, and integrating networking concepts with a graphical user interface.